



PYTHON MODULES, PACKAGES & LIBRARIES

MASTER CODE ORGANIZATION AND BUILD SCALABLE PYTHON APPLICATIONS

WHAT IS MODULE IN PYTHON?

- **The module is a simple Python file that contains collections of functions and global variables and with having a .py extension file.** It is an executable file and to organize all the modules we have the concept called Package in Python.
- **Example of Modules:** Datetime,Regex,Random etc.

WHAT IS PACKAGE IN PYTHON?

- The **package** is a simple directory having collections of modules. This directory contains Python modules and also having `__init__.py` file by which the interpreter interprets it as a Package. The package is simply a namespace. The package also contains sub-packages inside it.
- A **package** is a folder that contains multiple Python modules (and possibly sub-packages).

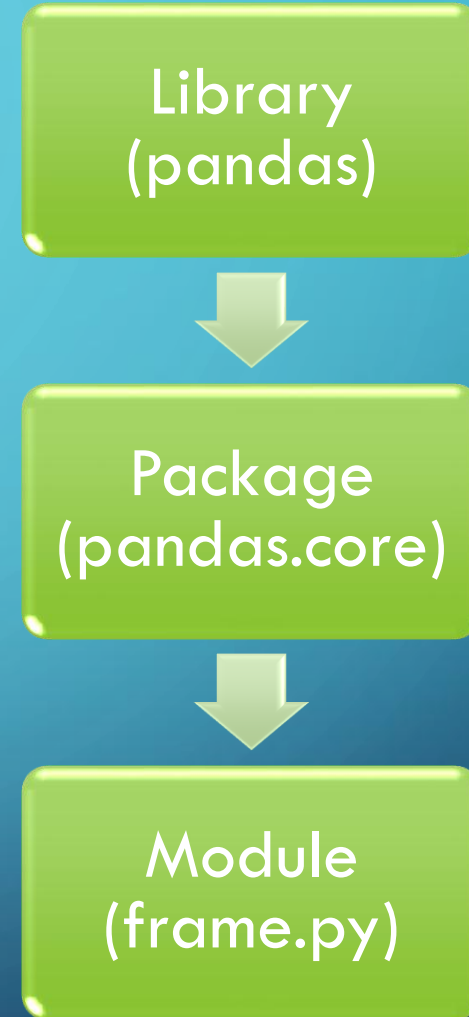
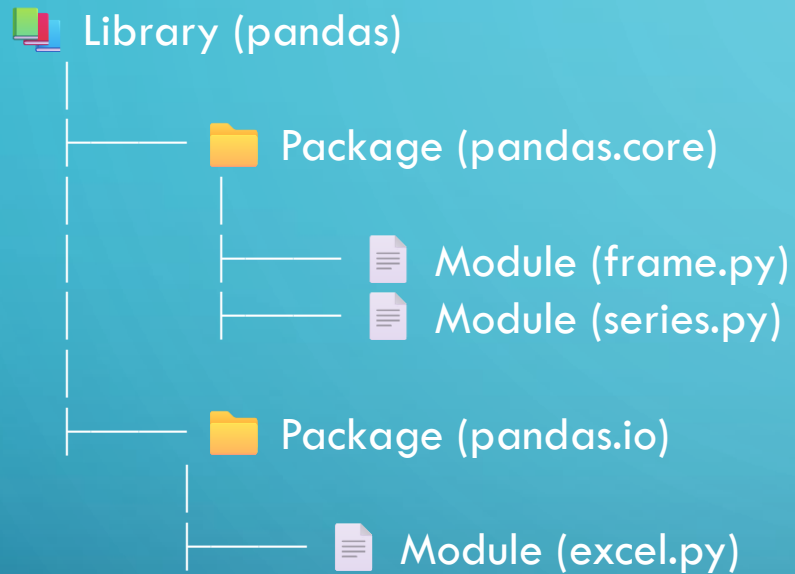
WHAT IS LIBRARY IN PYTHON

- The **library** is having a collection of related functionality of codes that allows you to perform many tasks without writing your code. **It is a reusable chunk of code that we can use by importing it into our program**, we can just use it by importing that library and calling the method of that library with a period(.). However, it is often assumed that while a package is a collection of modules, a library is a collection of packages.
- Example of Libraries: Matplotlib, Seaborn etc.

DIFFERENCE BETWEEN MODULE, PACKAGES, AND LIBRARY

Feature	Module	Package	Library
Definition	A single Python file containing code	A folder containing multiple modules	A collection of packages and modules for a broader purpose
Structure	.py file	Directory (folder)	Installable ecosystem
Purpose	Organize small reusable code	Organize related modules	Provide complete functionality for a domain
Contains	Functions, classes, variables	Modules and sub-packages	Multiple packages and modules
Import Style	import module_name	From package import module	import library_name
Creation	Create a .py file	Create folder + __init__.py	Usually installed via pip
Example	Math,random	Email,pandas.core	pandas, numpy, matplotlib

STRUCTURE



WHY DO WE NEED MODULES, PACKAGES, AND LIBRARIES?

The Core Problem (Without Them)

- ● Code becomes too large and messy
- ● Hard to maintain and debug
- ● Duplicate code everywhere
- ● Difficult team collaboration
- ● Poor scalability of applications
- ● Reinventing the wheel repeatedly

WHY WE NEED MODULES

- **Purpose:** Break large programs into smaller reusable files
- **Benefits:**
 - ✓ Improves code readability
 - ✓ Enables code reuse
 - ✓ Easier debugging
 - ✓ Better organization
 - ✓ Supports unit testing
- **Use when:** You want to split logic into manageable pieces.

WHY WE NEED PACKAGES

- **Purpose:** Organize related modules into structured folders
- **Benefits:**
 - ✓ Keeps large projects structured
 - ✓ Avoids naming conflicts
 - ✓ Supports hierarchical imports
 - ✓ Improves project scalability
 - ✓ Makes codebase professional
- **Use when:** Your project has many modules.

WHY WE NEED LIBRARIES

- **Purpose:** Provide ready-made solutions for common problems
- **Benefits:**
 - ✓ Saves development time
 - ✓ Avoids reinventing the wheel
 - ✓ Industry-tested and optimized
 - ✓ Speeds up development
 - ✓ Provides advanced capabilities
- **Use when:** You need powerful prebuilt functionality.
- **Note:** “Don’t write everything — organize what you write and reuse what already exists.”

BUILT-IN PYTHON LIBRARIES (STANDARD LIBRARY)

Built-in libraries are modules and packages that come pre-installed with Python — no pip install required.

- Data Handling & Formats: json, csv, xml
- Mathematical & Numeric: math, random, statistics, decimal
- File & OS Operations: os, sys, pathlib (**Used for:** file handling, system operations)
- Date & Time: datetime, time, calendar (**Used for:** date/time manipulation)

BUILT-IN VS EXTERNAL LIBRARIES

Feature	Built-in	External
Installation	Comes with Python	Requires pip
Example	math, os	Pandas,numpy
Maintenance	Python core	Third-party

WHAT ARE EXTERNAL LIBRARIES?

- External libraries are third-party packages that are **not included with Python** and must be installed separately.
- **Key Characteristics**
 - ✓ Not pre-installed with Python
 - ✓ Installed using pip
 - ✓ Developed by third-party contributors
 - ✓ Provide advanced functionality
 - ✓ Located in site-packages
- Installation: using pip command (pip install pandas)

HOW TO FIND MODULES INSIDE A PYTHON LIBRARY

METHOD 1 — USING DIR()

```
import pandas  
  
print(dir(pandas))
```

Displays:

- sub-packages
- modules
- functions
- classes

METHOD 2 — USING HELP()

```
import pandas  
  
help(pandas)
```

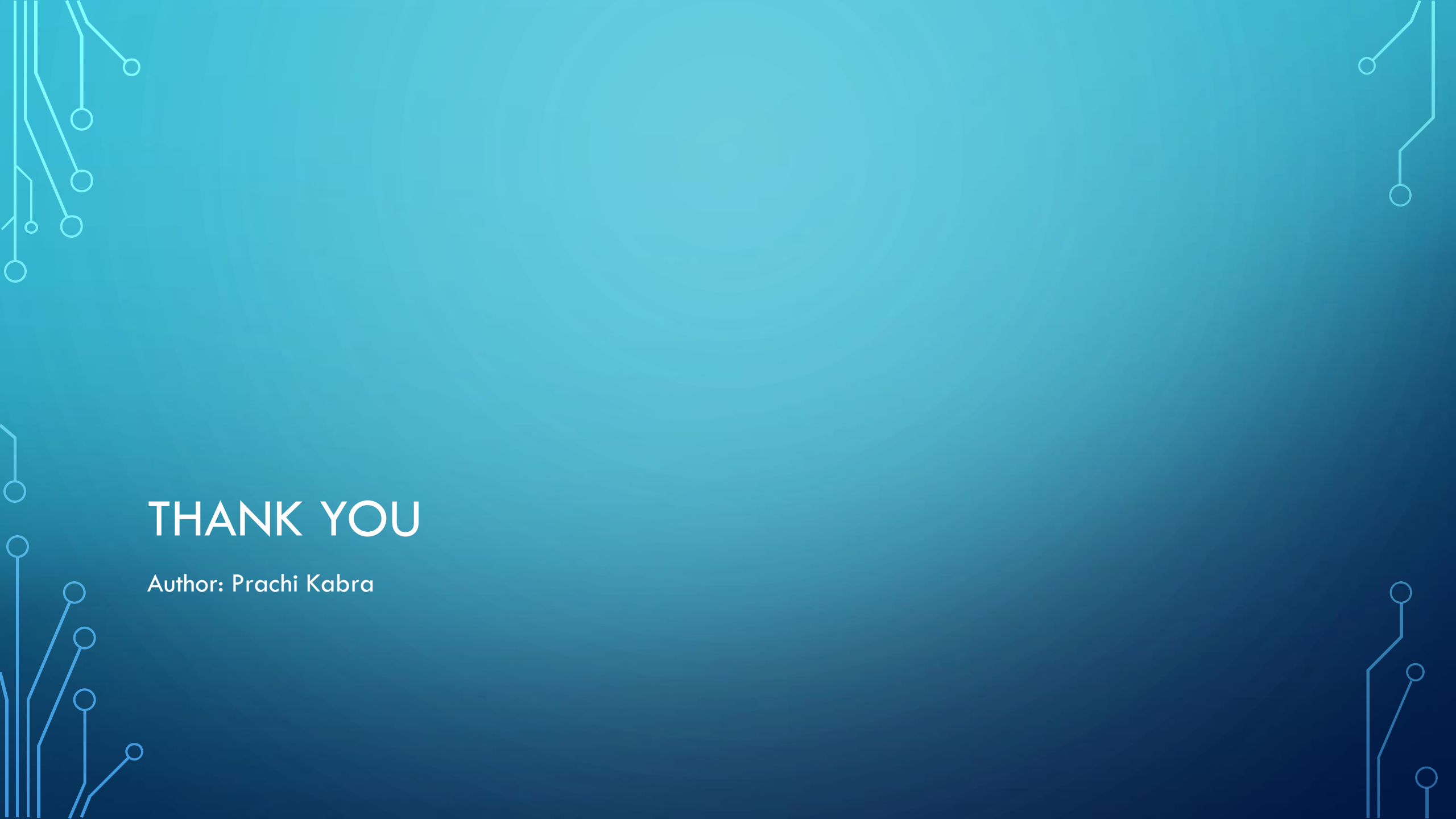
Shows:

- documentation
- available modules
- functions
- classes

COMMON COMMANDS

- `pip show pandas`
- `pip list`
- Find where is library installed:

```
import pandas  
  
print(pandas.__file__)
```

The background is a blue gradient with decorative white circuit-like lines in the corners. The lines consist of straight segments and small circles, resembling a stylized electronic circuit.

THANK YOU

Author: Prachi Kabra